Audio to MIDI

SMP Project Presentation

Peter Wienken Aaron Michael Weis

March 9, 2026

Agenda

1. Introduction and motivation
2. Software architecture
3. Audio preprocessing
4. STFT and time-frequency analysis
5. Spectral flux and onset detection
6. Peak picking
7. MIDI export
8. Conclusion and outlook

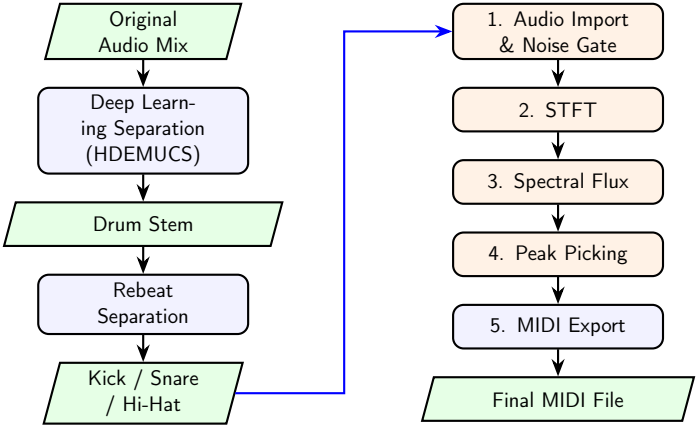
Project Goal and Motivation

- Goal: extract **kick**, **snare**, and **hi-hat** onsets from an audio track.
- Convert detected drum events into a **MIDI file**.
- This allows:
  - recreating grooves from songs,
  - editing rhythms in a DAW,
  - generating rhythmic transcriptions.
- Main challenge: accurate separation and onset detection in real audio.

Software and Libraries

- **Python** implementation in **VS Code**
- Main libraries:
  - **Librosa**: loading audio and STFT
  - **NumPy**: numerical operations
  - **Matplotlib**: visualization
  - **Mido**: MIDI export
  - **PyTorch / TorchAudio**: drum stem separation
- Additional tool:
  - **Rebeat**: separation into kick, snare and hi-hat

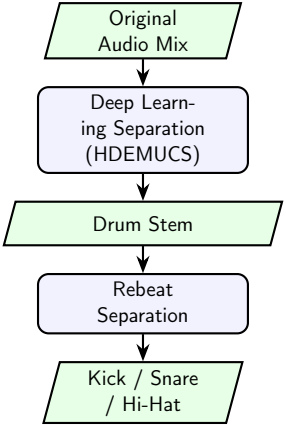
Overall Signal Processing Pipeline

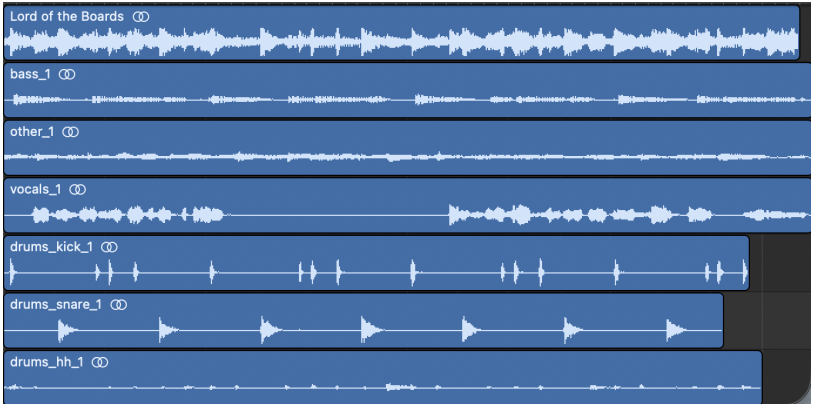


Object-Oriented Design

- Strict **object-oriented design**
- Each processing step is implemented in a dedicated class
- Advantages:
  - modular structure
  - easier debugging
  - no global variables
  - better maintainability
- Main classes:
  - `LoadAudioFile`
  - `LoadSTFT`
  - `SpectralFluxCalculator`
  - `PeakPicking`
  - `MidiExport`

Audio Separation





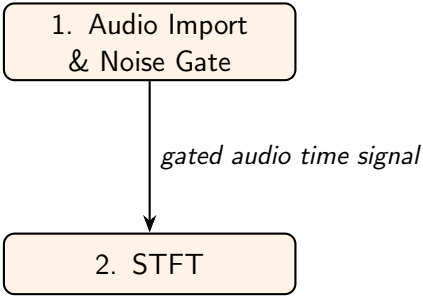
Audio Import and Noise Gate

- Kick / Snare / HiHat Audio is loaded with **librosa**
- Mono conversion and resampling to **44.1 kHz**
- A **noise gate** suppresses low-amplitude noise
- Threshold based on the **90th percentile** of the envelope

$$\text{env} = |\text{audio}| \qquad T = \text{percentile}(\text{env}, 90)$$

$$x_{\text{gated}}[n] = \begin{cases} x[n], & |x[n]| \geq T \\ 0, & |x[n]| < T \end{cases}$$

Audio Import to STFT



FFT and STFT

- The **FFT** efficiently computes the Discrete Fourier Transform.
- Computational complexity is reduced from

$$O(N^2) \rightarrow O(N \log N)$$

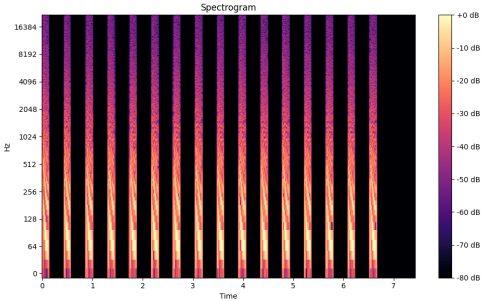
- For non-stationary audio, the signal is divided into overlapping frames.
- A window function is applied to each frame.
- This produces the **Short-Time Fourier Transform (STFT)**.

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N} \qquad |\mathrm{X_e}| = \begin{bmatrix} 1 & 4 & 1 & 6 \\ 3 & 2 & 2 & 1 \\ 1 & 5 & 3 & 6 \end{bmatrix}$$

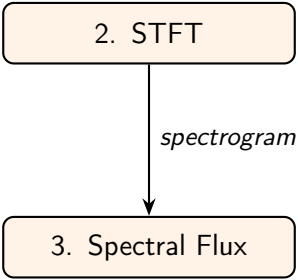
STFT: Principle and Spectrogram

- The signal is split into short overlapping time frames.
- Each frame is multiplied by a window function.
- Then an FFT is calculated for every frame.

- The result is a time-frequency representation: the **spectrogram**.
- Columns represent time frames.
- Rows represent frequency bins.
- This makes transient drum events visible over time.



STFT to Spectral Flux



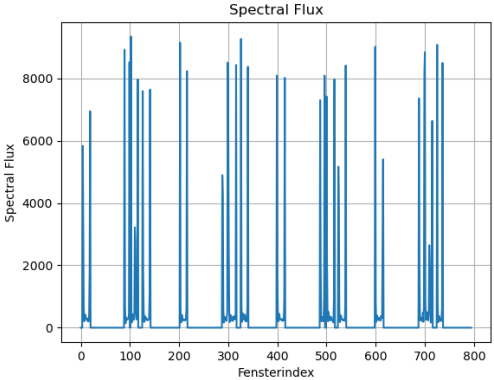
Spectral Flux

- Drum hits create sudden increases in spectral energy.
- These increases are measured by the **spectral flux**.
- First, the logarithmic magnitude is calculated:

$$L(k) = \log_{10}(|X(k)|)$$

$$\Delta L(k) = L_n(k) - L_{n-1}(k)$$

$$F(n) = \sum_k |\Delta L(k)|$$



Spectral Flux Example with Matrices

Given the logarithmic magnitude spectrum:

$$\mathsf{L} = \begin{bmatrix} 1 & 4 & 1 & 6 \\ 3 & 2 & 2 & 1 \\ 1 & 5 & 3 & 6 \end{bmatrix}$$

The difference between consecutive frames is:

$$\Delta\mathsf{L} = \begin{bmatrix} 4 & 1 & 6 \\ 2 & 2 & 1 \\ 5 & 3 & 6 \end{bmatrix} - \begin{bmatrix} 1 & 4 & 1 \\ 3 & 2 & 2 \\ 1 & 5 & 3 \end{bmatrix} = \begin{bmatrix} 3 & -3 & 5 \\ -1 & 0 & -1 \\ 4 & -2 & 3 \end{bmatrix}$$

Summing the absolute values column-wise gives:

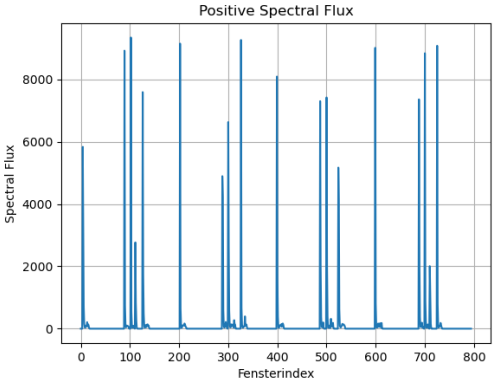
$$\mathsf{F} = \begin{bmatrix} |3| + |-1| + |4| \\ |-3| + |0| + |-2| \\ |5| + |-1| + |3| \end{bmatrix} = \begin{bmatrix} 8 \\ 5 \\ 9 \end{bmatrix}$$

Half-Wave Rectification

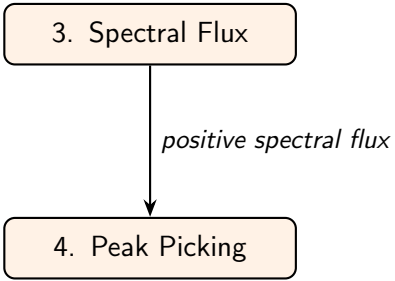
- Only **positive** energy changes are relevant.
- Negative values are discarded.

$$F_{\text{rect}}(n) = \max(0, F(n))$$

- This emphasizes attack transients.
- It improves onset detection for percussive sounds.



Spectral Flux to Peak Picking



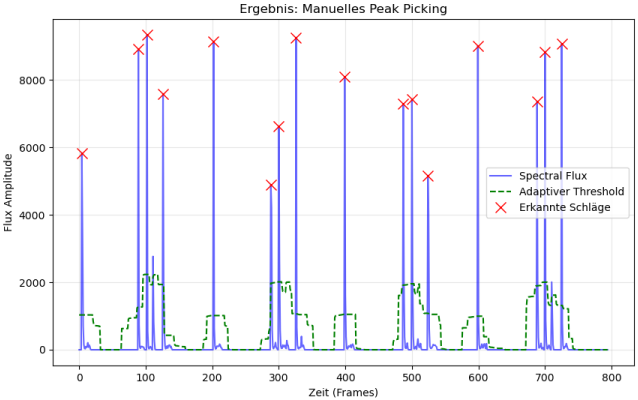
Adaptive Threshold and Peak Picking

- A fixed threshold is not robust enough for varying audio levels.
- Therefore, an **adaptive threshold** is used.
- It is based on a moving average:

$$y[n] = \frac{1}{N} \sum_{k=-\lfloor N/2 \rfloor}^{\lfloor (N-1)/2 \rfloor} x[n+k]$$

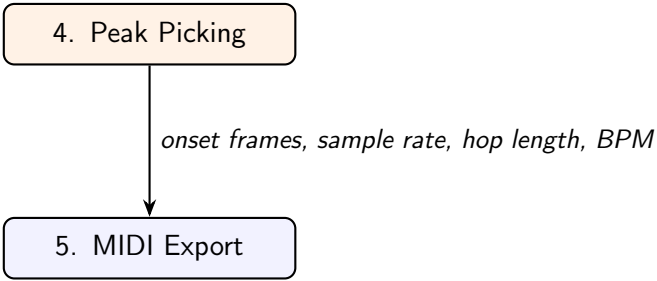
- An additional offset avoids detections from the noise floor.
- A valid onset must satisfy:
  1. above threshold,
  2. local maximum,
  3. enough distance to the previous onset.

Peak Picking Visualization



- Peaks are selected only if they exceed the adaptive threshold.
- Debouncing prevents multiple detections of the same hit.

Peak Picking to MIDI Export



MIDI Export

- Detected onset frames are converted into MIDI events.
- Required parameters:
  - onset frames,
  - sample rate,
  - hop length,
  - BPM.
- Output:
  - **multi-track MIDI file**
  - routed to **MIDI channel 10**
- General MIDI mapping:
  - Kick: 36
  - Snare: 38
  - Closed Hi-Hat: 42

Conclusion and Outlook

- The system can generate usable MIDI drum patterns from audio.
- It combines:
  - AI-based source separation
  - classical DSP methods
  - rule-based onset detection
- Main limitations:
  - imperfect source separation
  - overlap between drum components
  - noise in extracted stems
- Future work:
  - improved separation models
  - more robust onset detection
  - automatic tempo estimation

References

- Librosa Developers, *Librosa documentation*. https://librosa.org/doc/main/index.html
- Matplotlib Developers, *Matplotlib Documentation*. https://matplotlib.org/
- NumPy Developers, *NumPy Documentation*. https://numpy.org/devdocs/index.html
- PyTorch Developers, *PyTorch Documentation*. https://pytorch.org/
- Rebeat Developers, *Rebeat App and documentation*. https://rebeatapp.com/
- SciPy Developers, *Scipy Documentation*. https://docs.scipy.org/doc/scipy/index.html
- Torchaudio Developers, *Hybrid Demucs Tutorial*. https://docs.pytorch.org/audio/2.8/tutorials/hybrid_demucs_tutorial.html
- Mido Developers, *Mido Documentation*. https://mido.readthedocs.io/en/stable/

Thank you for your attention